

Social Informatics

Technical Report

Eye Disease Classification System

Team Name : Maybe!

Team Members :

Salma Yasser Mostafa

Ola Farhat Ibrahim

Nouran Atef Hashad

Nermeen Mohamed

Supervisors :

Dr. Asmaa Saad

Eng. Yasmeen Elfike

Table of Contents

1. Project Overview
2. Dataset Description
3. Data Preprocessing
4. Machine Learning Workflow
5. Deep Learning Workflow
6. Results and Model Comparison
7. Comparison with Kaggle Reference Notebook
8. Deployment Workflow
9. Conclusion
10. Output Screenshots

1. Project Overview

This project aims to build an eye disease classification system using retinal fundus images. The model classifies each image into one of four classes: cataract, diabetic retinopathy, glaucoma, or normal. The work includes dataset exploration, preprocessing, training six models, evaluating the models using medical classification metrics, and deploying the selected model as a Streamlit web application inside a Docker container on Hugging Face Spaces.

The system is designed as both a research experiment and a deployable prototype. The research notebook compares traditional machine learning and deep learning approaches, while the deployed application uses the best practical model for fast CPU inference.

Project Objectives

- Understand and preprocess the retinal image dataset.
- Train three traditional machine learning models using HOG features.
- Train three deep learning models directly on images.
- Compare all models using accuracy, precision, recall, F1-score, and confusion matrix.
- Select the best model for deployment based on performance and deployment suitability.
- Build a GUI using Streamlit and deploy it online using Docker and Hugging Face Spaces.

2. Dataset Description

The dataset contains retinal images divided into four classes: cataract, diabetic retinopathy, glaucoma, and normal. Each class contains approximately one thousand images. The dataset is based on retinal fundus images collected from different public sources used for eye disease recognition tasks, such as IDRiD, ocular recognition sources, and HRF-like retinal image collections.

Class Distribution

Class	Number of Images
cataract	1038
diabetic_retinopathy	1098
glaucoma	1007
normal	1074

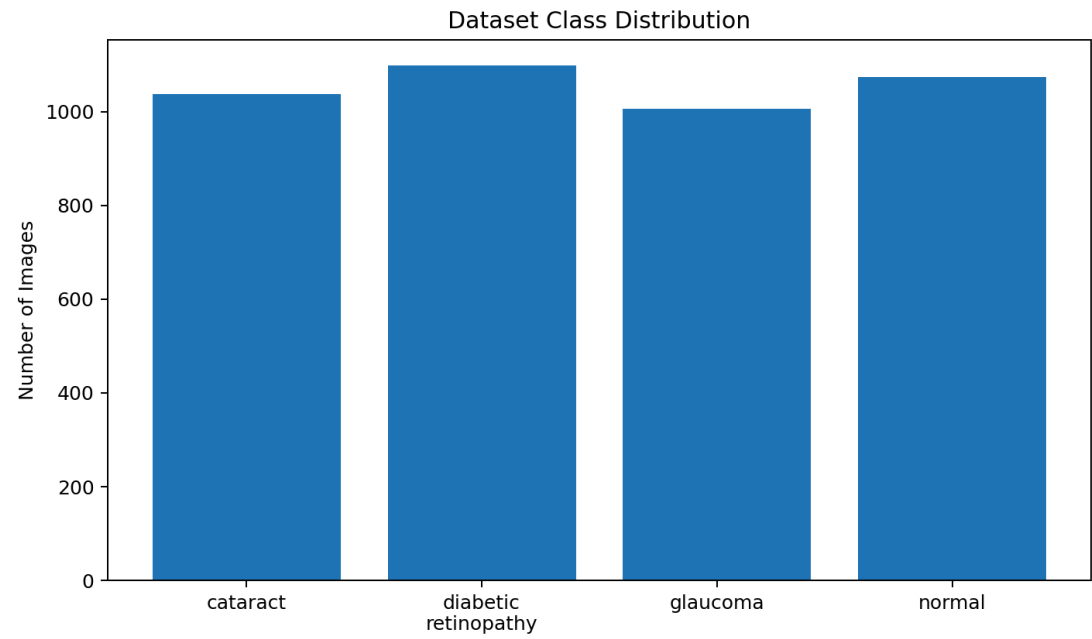


Figure 1. Dataset class distribution.

Dataset Split

Class	Train	Validation	Test
cataract	726	155	157
diabetic_retinopathy	768	164	166
glaucoma	704	151	152
normal	751	161	162

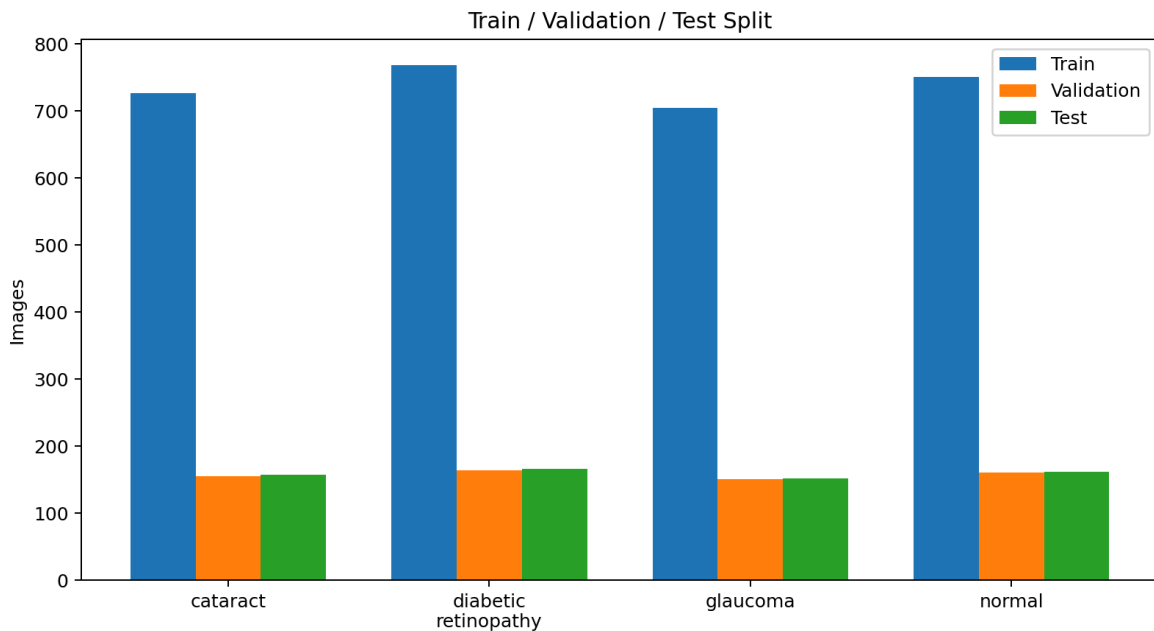


Figure 2. Train/validation/test split distribution.

The dataset was split into 70% training, 15% validation, and 15% testing. This split provides a separate validation set for tuning and a final unseen test set for fair model evaluation.

3. Data Preprocessing

The preprocessing steps differed slightly between traditional machine learning and deep learning models. For machine learning, HOG features were extracted from grayscale resized images. For deep learning, images were resized and augmented using ImageDataGenerator.

Common Dataset Checks

- Counted images in each class to check class balance.
- Visualized random images from each class to understand image appearance.
- Checked image sizes to confirm the need for resizing.
- Checked corrupted images using PIL image verification. No corrupted images were found.
- Created train, validation, and test folders using splitfolders.

Machine Learning Preprocessing

- Resize each image to 128×128 pixels.
- Convert the image to grayscale.
- Extract HOG features using 9 orientations, 8×8 pixels per cell, and 2×2 cells per block.

- Scale HOG features using StandardScaler.
- Encode class names using LabelEncoder.

Deep Learning Preprocessing

- Resize images to 128×128 for Lightweight CNN to reduce CPU load.
- Resize images to 160×160 for MobileNetV2 and EfficientNetB0 transfer learning.
- Apply augmentation such as rotation, zoom, shift, and horizontal flip.
- Use model-specific preprocessing for MobileNetV2 and EfficientNetB0.

4. Machine Learning Workflow

Traditional machine learning models cannot automatically learn spatial features from images like CNNs. Therefore, handcrafted HOG features were extracted first and then used as input to classical classifiers.

Deployment Prediction Pipeline

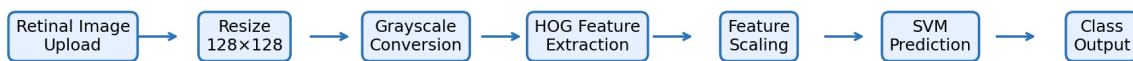


Figure 3. SVM + HOG deployment prediction pipeline.

Models Implemented

Model	Feature Extraction	Notes
Logistic Regression	HOG	Linear baseline classifier with scaled features.
SVM	HOG	Best performing model using RBF kernel.
Random Forest	HOG	Tree-based ensemble model with balanced class weights.

The best traditional machine learning model was SVM + HOG with 85.71% test accuracy and weighted F1-score of 0.85.

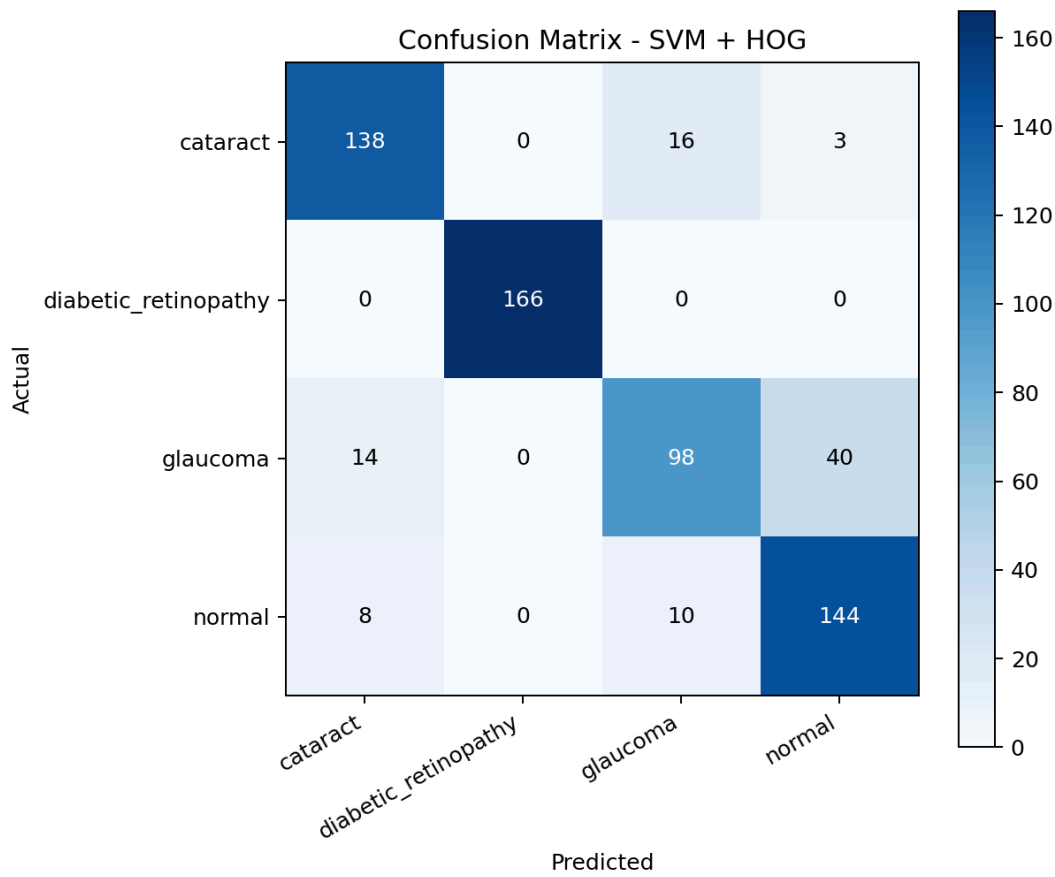


Figure 4. Confusion matrix for SVM + HOG.

5. Deep Learning Workflow

Three deep learning models were trained and evaluated. The first model was a lightweight CNN trained from scratch. The second and third models used transfer learning with MobileNetV2 and EfficientNetB0. Due to running on native Windows CPU, the models were configured with smaller image sizes and batch sizes to reduce computational load.

Model	Image Size	Approach	Result Summary
Lightweight CNN	128 × 128	CNN from scratch with GlobalAveragePooling2D	Low performance; used as DL baseline.
MobileNetV2	160 × 160	Transfer learning with frozen base model	Better than CNN but below SVM.
EfficientNetB0	160 × 160	Transfer learning with frozen base model	Similar to MobileNetV2; better glaucoma recall.

The lightweight CNN achieved only 52.43% accuracy, showing that a simple CNN trained from scratch was not sufficient. MobileNetV2 achieved 75.98%, while EfficientNetB0 achieved 75.51%.

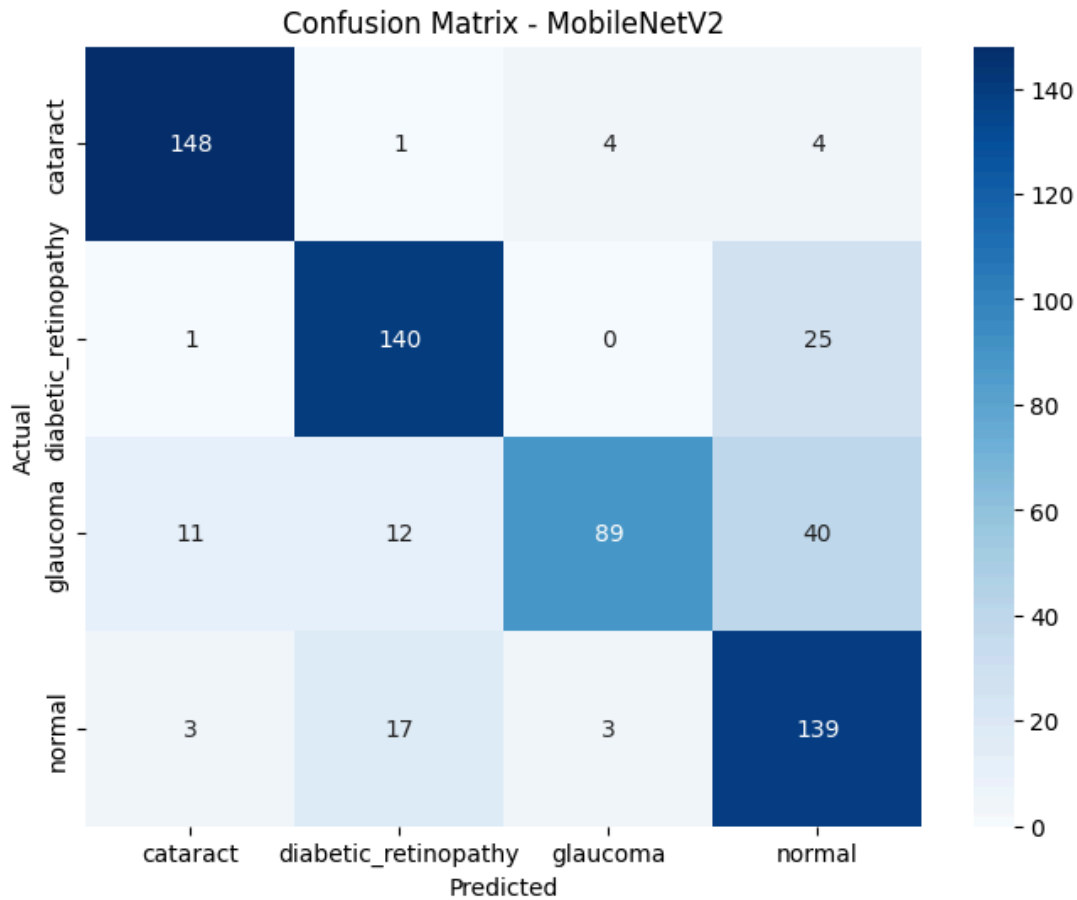


Figure 5. Example confusion matrix from MobileNetV2 experiment.

6. Results and Model Comparison

Model	Type	Test Accuracy	Weighted F1	Glaucoma Recall
Logistic Regression + HOG	Machine Learning	81.95%		
SVM + HOG	Machine Learning	85.71%	0.85	0.64
Random Forest + HOG	Machine Learning	81.79%		
Lightweight CNN	Deep Learning	52.43%	0.47	0.03
MobileNetV2	Deep Learning	75.98%	0.75	0.49
EfficientNetB0	Deep Learning	75.51%	0.76	0.62

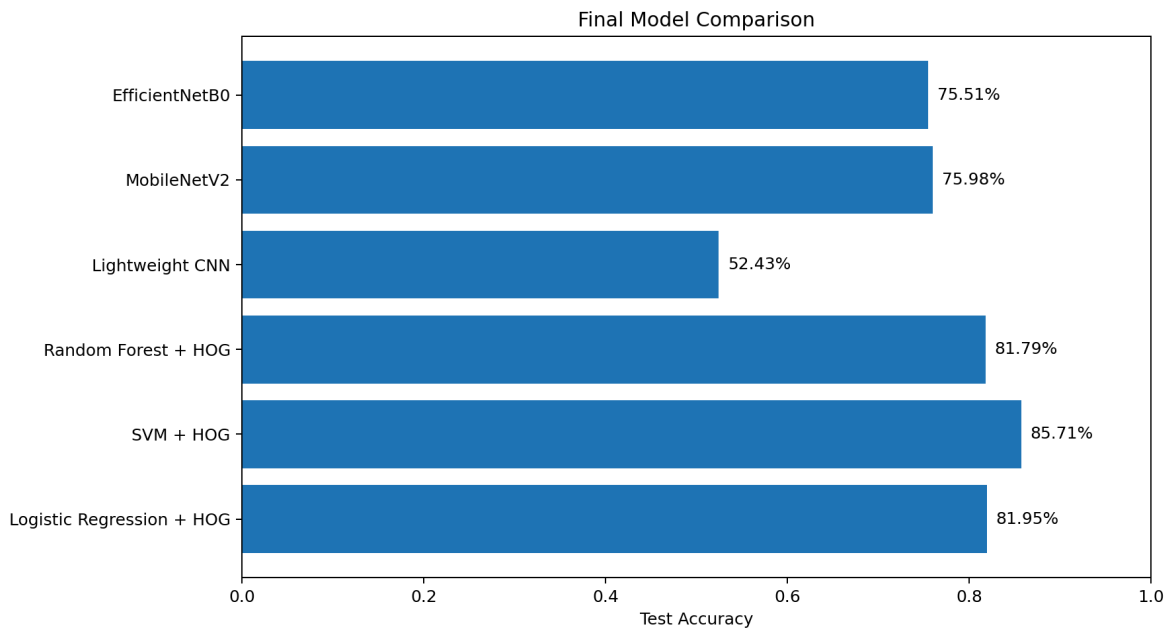


Figure 6. Final comparison between machine learning and deep learning models.

The SVM + HOG model achieved the best overall test accuracy of 85.71%. It also had the best practical deployment suitability because it is lightweight, fast on CPU, and does not require TensorFlow during inference.

Medical Interpretation

Across several models, **glaucoma** was the most challenging class. Many glaucoma images were **misclassified as normal**. This is medically important because missing a diseased case is more critical than producing a false alarm. Therefore, glaucoma recall was considered an important metric during model comparison.

7. Comparison with Kaggle Reference Notebook

The Kaggle reference notebook used EfficientNetB3 as a transfer learning model. It used an 80/10/10 split, image size 256 × 256, data augmentation including rotation, horizontal flip, vertical flip, and brightness adjustment, and trained EfficientNetB3 for 15 epochs. The reported final training accuracy was approximately 80.97% and validation accuracy was approximately 81.99%.

Improvements Added in Our Work

- Implemented a full comparison between six models instead of using one model only.
- Added traditional ML baselines using HOG features: Logistic Regression, SVM, and Random Forest.
- Added deep learning baselines: Lightweight CNN, MobileNetV2, and EfficientNetB0.

- Evaluated models on an independent test set using classification report and confusion matrix.
- Focused on medical metrics such as recall, especially for glaucoma.
- Selected the deployment model based on accuracy, F1-score, model size, CPU speed, and deployment practicality.
- Built a Streamlit GUI and deployed the selected model using Docker on Hugging Face Spaces.

Although the Kaggle notebook achieved strong validation accuracy using EfficientNetB3, our work added a broader experimental workflow and showed that SVM + HOG achieved the best test accuracy in our setup. This made SVM + HOG the most suitable model for deployment.

8. Deployment Workflow

The final deployed system uses the SVM + HOG model. The application was built with Streamlit, containerized with Docker, pushed to GitHub using Git LFS for large model files, and deployed on Hugging Face Spaces as a Docker Space.

Deployment Components

File / Folder	Purpose
app.py	Streamlit GUI and prediction logic.
requirements.txt	Python dependencies required by the app.
Dockerfile	Defines the reproducible container environment.
README.md	Hugging Face Space metadata and project overview.
saved_models/svm_hog.pkl	Trained SVM model.
saved_models/hog_scaler.pkl	Scaler used to normalize HOG features.
saved_models/label_encoder.pkl	Class label encoder.
results/final_models_comparison.csv	Final comparison table shown in the app.
reports/svm_report.txt	Classification report for the selected model.

The prediction flow in the deployed app is: image upload, resize to 128 × 128, grayscale conversion, HOG feature extraction, feature scaling, SVM prediction, then displaying the predicted disease class in the GUI.

9. Conclusion

This project implemented a complete workflow for eye disease classification. The final system compared six machine learning and deep learning models. SVM + HOG achieved the highest test accuracy and was selected for deployment because it is accurate, lightweight, and efficient on CPU. The final prototype demonstrates both model experimentation and real deployment through Streamlit, Docker, GitHub, and Hugging Face Spaces.

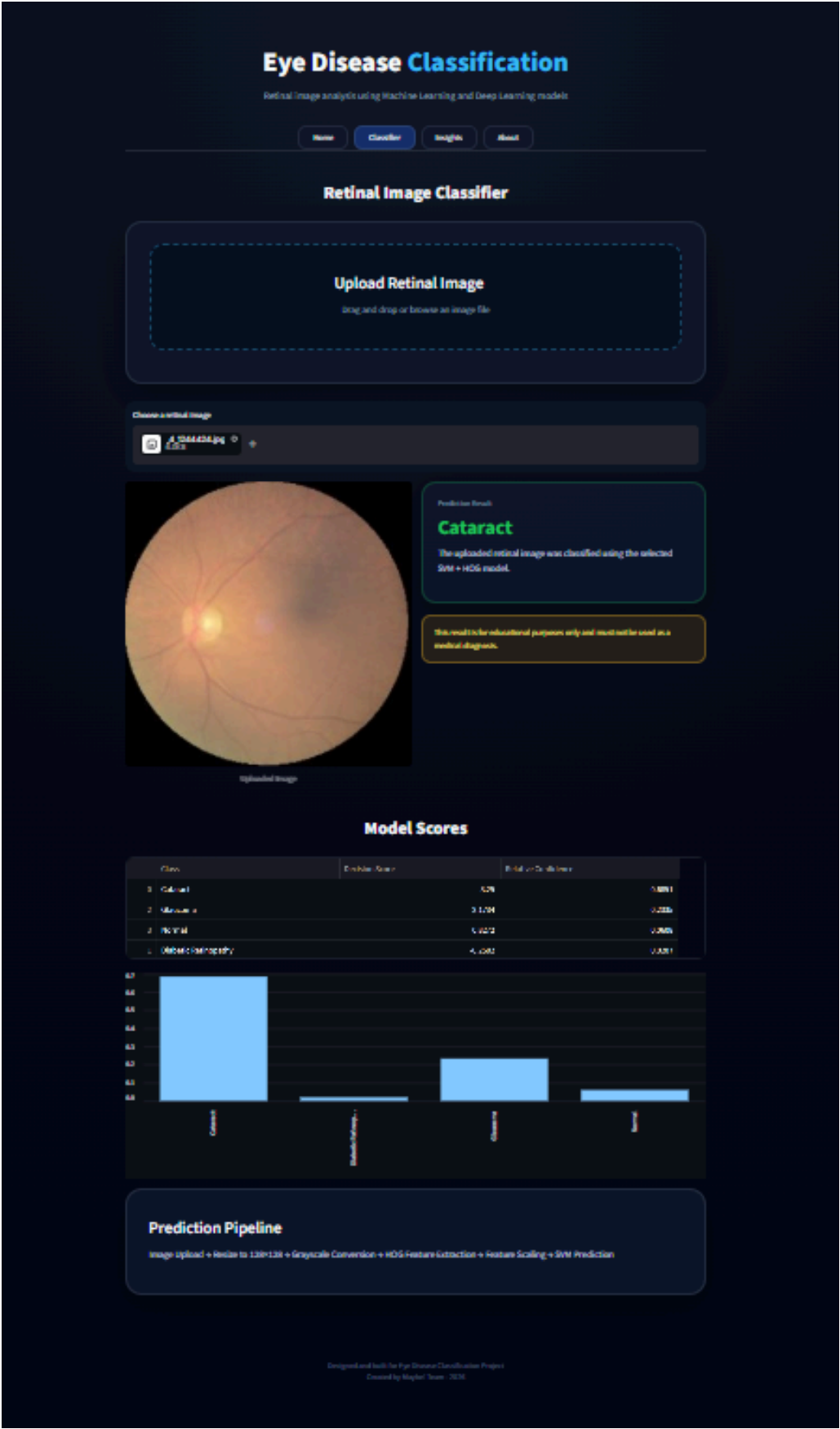
10. Output Screenshots

This section includes selected screenshots and visual outputs from the project implementation, model evaluation, and deployment process.

<https://salmayasser-eye-disease-classification.hf.space/>



Figure 7.Home





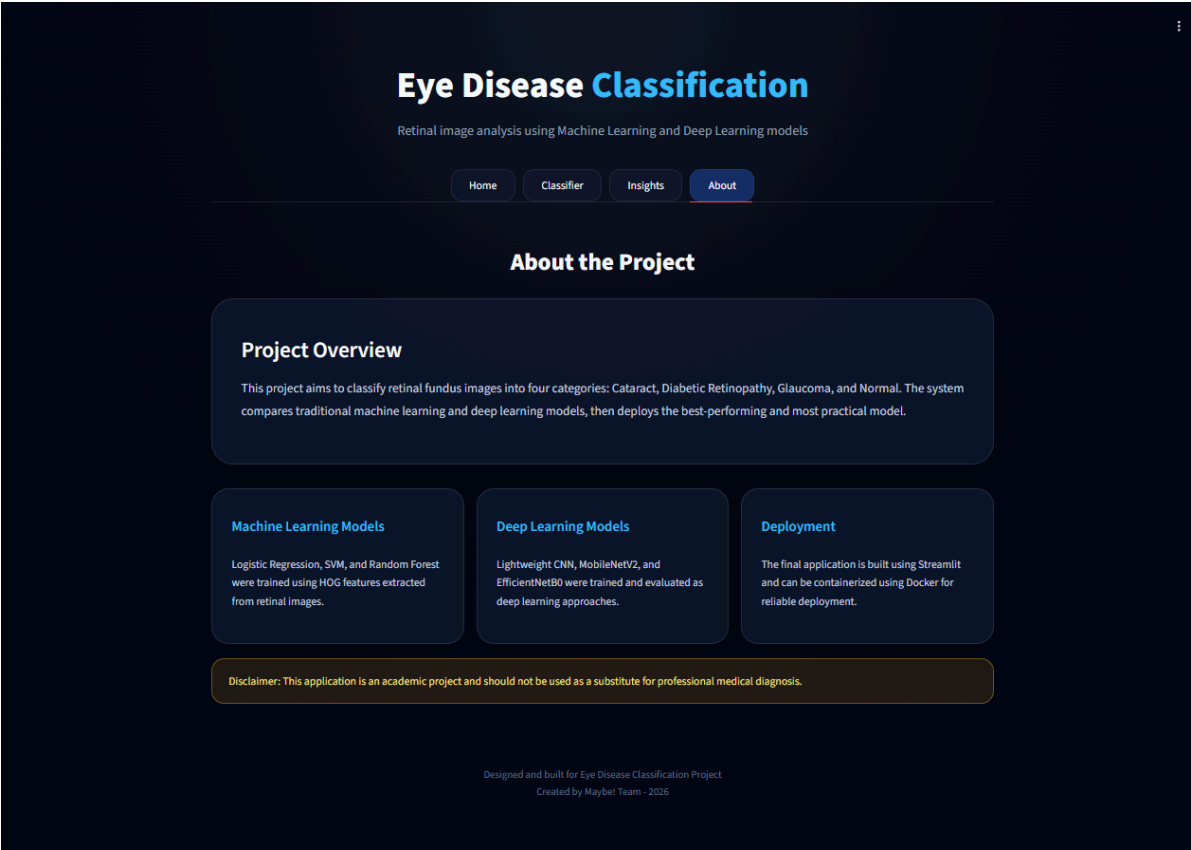


Figure 10.About

References

- Kaggle reference notebook: Eye Diseases Classification by Mahnaz Arjmand.
- Kaggle dataset: Eye Diseases Classification Dataset.
- Streamlit documentation for web app development and deployment.
- Hugging Face Spaces documentation for Docker Spaces deployment.